

总目标

对每个笑容片段（一个子文件夹）：

- 提取特征序列
- 做 **baseline** 对齐
- 计算动态指标
- 进行时间归一化
- 构建每类的典型轨迹（**prototype**）
- 计算类间差分向量
- 生成分析结果与可视化

研究核心：

不同笑容类型在特征空间中的时间演化轨迹差异

数据结构假设

```
"E:\Matsuda_data\2-18meeting"/
├─ polite/
│   ├── 1/
│   ├── 2/
│   └─ ...
├─ truesmile/
│   ├── 0/
│   └─ ...
└─ ambiguous/
    ├── 0/
    └─ ...
```

每个子文件夹：

```
20250926_59760-59856_from_59743-59958_0_0_89.png
20250926_59760-59856_from_59743-59958_0_0_90.png
...
```

排序标准以最后的数字为准，例如89，90，91

从文件名末尾提取连续数字作为帧编号 按数值排序

注：可能存在跳跃的问题 帧号不连续允许 按排序顺序视为时间序列 不做补帧 **videos** / **video** 目录不参与本流程分析

帧率：30 fps

对于每一个片段：

STEP 1 — 特征提取

模型：

VGG-Face fc7

模型使用方法和提取方法可以参考：“E:\Single_frame_smile\Analysis\analysis2-12”中的脚本和log还有summary。

fc7 D应该是4096 d

对每一帧：

$$f(t) \in \mathbb{R}^D$$

保存：

```
sequence_features.npy
```

结构：

```
[T, D]
```

注意文件和特征向量的对应

STEP 2 — Baseline 对齐

定义：

$$f_0 = \text{mean}(f(0:5))$$

前 5 帧平均作为 neutral baseline。

计算相对特征：

$$f_{\text{rel}}(t) = f(t) - f_0$$

保存：

```
sequence_features_rel.npy
```

⚙️ STEP 3 — 表情强度曲线 (magnitude)

计算：

```
d(t) = || f_rel(t) ||
```

得到：

```
distance curve
```

指标：

- peak magnitude
- mean magnitude
- std magnitude

保存：

```
distance_curve.npy  
metrics.json
```

⚙️ STEP 4 — 变化速率 (velocity)

计算：

```
v(0) = 0  
v(t) = || f_rel(t) - f_rel(t-1) ||
```

得到：

```
velocity curve
```

指标：

- mean velocity
- peak velocity
- total motion energy

保存：

```
velocity_curve.npy  
metrics.json
```

⚙ STEP 5 — Duration (时长)

定义：

```
duration_frames = T  
duration_seconds = T / 30
```

统计：

每类平均 duration (秒) 。

保存：

```
duration_stats.csv
```

⚙ STEP 6 — 时间归一化 (关键)

目的：

不同长度序列对齐。

方法：

把每个序列重采样到固定长度：

为了可视化和对齐检查

重采样结果单独生成一个文件夹，并且按照采样顺序复制并重命名 (000.png 到 019.png) 。

保证采样的图片和特征向量对齐 重采样图片目录结构与源数据保持一致 (class/sequence 两级结构)

所有归一化基于 baseline-aligned feature

重采样：

```
N = 20
```

得到：

```
f_norm(t)    shape = [20, D]
d_norm(t)    shape = [20]
v_norm(t)    shape = [20]
```

插值方法：

线性插值。

保存：

```
normalized_sequence.npy
```

STEP 7 — 每类典型轨迹 (Prototype)

找到每一类的一个最典型例子

Prototype Trajectory Methods — Mathematical Formulation

Unified Notation

- Class: $c \in \{\text{polite}, \text{truesmile}, \text{ambiguous}\}$
- Sequence index: $i = 1, \dots, N_c$
- Time-normalized length: $T = 20$
- Feature dimension: D
- Baseline-aligned and time-normalized feature sequence:

$$\mathbf{f}_i(t) \in \mathbb{R}^D, \quad t = 1, \dots, T$$

Method A: Median Trajectory (Required)

Definition

For class c , at each time point t and feature dimension d , compute the median across sequences:

\$\$ \mathbf{p}_c(t)_d

$\operatorname{median}_{i=1,\dots,N_c} \left(\mathbf{f}_i(t)_d \right)$

Equivalently:

\$\$ \mathbf{p}_c(t)

$\operatorname{median}_i \left(\mathbf{f}_i(t) \right)$

The median is computed **element-wise (per dimension)**.

Vector Form

Let

\$\$ F_c(t)

$\begin{bmatrix} \mathbf{f}_1(t) \ \mathbf{f}_2(t) \ \vdots \ \mathbf{f}_{N_c}(t) \end{bmatrix} \in \mathbb{R}^{N_c \times D}$

Then

\$\$ \mathbf{p}_c(t)

$\operatorname{median}_{\text{row}} \left(F_c(t) \right)$

Result

The prototype trajectory is:

\$\$ \mathbf{p}_c

$\{\mathbf{p}_c(1), \dots, \mathbf{p}_c(T)\} \in \mathbb{R}^{T \times D}$

Advantages

- Robust to outliers
 - Stable
 - Most commonly used in practice
-

Method B: K-Medoid Trajectory (Optional)

Objective

Select a real sequence that minimizes total distance to all other sequences:

$$\sum_i \sum_{j=1}^{N_c} d(\mathbf{f}_i, \mathbf{f}_j)$$

The prototype is defined as:

$$\mathbf{p}_c(t)$$

$$\mathbf{f}_{i^*}(t)$$

Distance Function (Key Component)

A distance between two sequences must be defined.

Euclidean Trajectory Distance

$$d(\mathbf{f}_i, \mathbf{f}_j)$$

$$\sum_{t=1}^T \|\mathbf{f}_i(t) - \mathbf{f}_j(t)\|_2$$

$$\text{or the average version:}$$

$$d(\mathbf{f}_i, \mathbf{f}_j)$$

$$\frac{1}{T} \sum_{t=1}^T \|\mathbf{f}_i(t) - \mathbf{f}_j(t)\|_2$$

$$\text{or the average version:}$$

Matrix Form

If

$$\mathbf{F}_i \in \mathbb{R}^{T \times D}$$

then:

$$d(F_i, F_j)$$

$$\|F_i - F_j\|_F$$

where $\|\cdot\|_F$ is the Frobenius norm.

Recommended Implementation (Most Stable)

Use Frobenius distance:

$$d(F_i, F_j)$$

$$\sqrt{\sum_{t=1}^T \|\mathbf{f}_i(t) - \mathbf{f}_j(t)\|_2^2}$$

$$\|\mathbf{f}_i(t) - \mathbf{f}_j(t)\|_2^2$$

This is equivalent to the Frobenius norm of the difference matrix.

Output

For each class:

$$\mathbf{p}_c \in \mathbb{R}^{T \times D}$$

保存：

```
prototype_polite.npy
prototype_truesmile.npy
prototype_ambiguous.npy
```

最终报告主要使用 median prototype medoid作为补充

⚙️ STEP 8 — 类间差分向量

计算：

```
Δ_polite_truesmile(t) = prototype_polite(t) - prototype_truesmile(t)
Δ_polite_amb(t)
Δ_truesmile_amb(t)
```

每个：

```
[20, D]
```

用途：

分析不同笑容类型在特征空间的方向差异。

保存：

```
class_difference_vectors.npy
```

结构：

```
{  
  "polite_vs_truesmile": [20, D],  
  "polite_vs_ambiguous": [20, D],  
  "truesmile_vs_ambiguous": [20, D]  
}
```

⚙️ STEP 9 — 时间段差分向量 (trajectory velocity field)

时间节点：

```
0%, 5%, 10%, ... 100%
```

计算：

```
segment_vector(k) = f_norm(t_{k+1}) - f_norm(t_k)
```

仅计算并保存每类 prototype 的 segment vectors。

保存：

```
segment_vectors.npy
```

⚙️ STEP 10 — 投影分析 (重要)

对于normalized的数据

使用：

PCA (默认)

目标：

把轨迹投影到 2D。按“每个时间点投影后连线”的方式展示轨迹时间变化。

需要：

- 所有样本轨迹
- prototype轨迹

输出：所有的图像 只有polite的case, polite 的prototype高亮表示

只有ambiguous的cases · 加上ambiguous的prototype的高亮表示

只有truesmile的加上 truesmile 的prototype

然后三种prototype的图像

图像：

```
trajectory_plot.png
trajectory_plot_polite.png
trajectory_plot_ambiguous.png
trajectory_plot_truesmile.png
trajectory_plot_cross.png
```

STEP 11 — 类间差异随时间变化

计算：

```
diff_norm(t) = || prototype_A(t) - prototype_B(t) ||
```

目标：

找到：

最大差异时间点。

保存：

```
class_distance_curve.csv
```

STEP 12 — Projection Score (高级)

对于每个样本：

计算：

```
score(t) = dot(f_norm(t), Δ_class_pair(t))
```

得到：

三组轴上的投影分数 (polite-vs-truesmile / polite-vs-ambiguous / truesmile-vs-ambiguous) 。

保存：

```
projection_scores.csv
```

STEP 13 — 可视化输出

必须生成：

1

平均 magnitude 曲线 (每类)

2

平均 velocity 曲线

3

prototype trajectory 2D 图

4

类间距离随时间曲线

5

duration 分布图

格式：

PNG + CSV

STEP 14 — 数据报告

生成：

```
dataset_report.csv
```

包含：

```
class
sequence_id
frames
duration_sec
peak_magnitude
mean_velocity
```

数学符号统一

```
f(t)      : 原始特征
f0         : baseline
f_rel(t)   : 相对特征
d(t)       : magnitude
v(t)       : velocity
```

Agent 交付内容

输出目录在：“E:\Matsuda_data\2-27meeting”

分析代码在：E:\Single_frame_smile\Analysis\analysis_sequence

最终需要：

```
"E:\Matsuda_data\2-27meeting"/
├─ prototypes/
├─ metrics/
├─ plots/
├─ csv/
└─ report/
```

补充约定：所有中间结果与最终结果均保存到 `E:\Matsuda_data\2-27meeting` 下对应子目录。